

Protocolo de Comunicación

1 - Estructura de los mensajes

Todos los mensajes intercambiados entre el cliente y el servidor se encapsularán en un objeto JSON. Para facilitar el procesamiento a través de los Sockets de Java, cada mensaje se enviará en una sola línea y finalizará con un carácter de salto de línea (\n).

Para el transporte, se utilizará el protocolo TCP con la codificación UTF-8. El modelo será de tipo: request/response + eventos asíncronos.

Formato base:

```
{  
  "command": "NOMBRE_DEL_COMANDO",  
  "information": {...}  
}
```

Formato de un mensaje (de un usuario no de las aplicaciones):

```
{  
  "room": "IA",  
  "sender": "nombre",  
  "timestamp": "2026-05-04T14:30:00",  
  "content": "Contenido del mensaje..."  
}
```

2 - Diccionario de Comandos

Comando	Descripción	Emisor	Estructura de los datos	Respuesta
REGISTER	Solicitar la creación de una cuenta.	Cliente	{ "user": "nombre" }	{ "type": "OK", "message": "password" } { "type": "ERROR", "message": "USERNAME_EXISTS" }
LOGIN	Iniciar sesión en el sistema.	Cliente	{ "user": "nombre", "key": 12345 }	{ "type": "OK", "message": "LOGIN_SUCCESS" }

				<pre>{“type”: “ERROR”, “message”: “INVALID_CREDENTIALS”}</pre>
<i>LIST_ROOMS</i>	Enumerar los salones existentes.	Cliente	<pre>{}</pre>	<pre>{“type”: “OK”, “message”: “NAME_ROOMS”} {“type”: “ERROR”, “message”: “NO_ROOMS_AVAILABLE”}</pre>
<i>SEND_ROOM_MSG</i>	Enviar un mensaje a un salón.	Cliente	<pre>{“room”: “IA”, “content”: “Hola”}</pre>	<pre>{“type”: “OK”, “message”: “MESSAGE_SENT”} {“type”: “ERROR”, “message”: “MESSAGE_TOO_LONG”} {“type”: “ERROR”, “message”: “NOT_IN_ROOM”}</pre>
<i>SEND_PRIV_MSG</i>	Enviar un mensaje privado.	Cliente	<pre>{“receiver”: “nombre2”, “content”: “Hola”}</pre>	<pre>{“type”: “OK”, “message”: “MESSAGE_SENT”} {“type”: “ERROR”, “message”: “USER_NOT_CONNECTED”} {“type”: “ERROR”, “message”: “USER_NOT_FOUND”}</pre>
<i>JOIN_ROOM</i>	Enviar un mensaje cuando un usuario entra en un salón.	Cliente	<pre>{“room”: “IA”}</pre>	<pre>{“type”: “OK”, “message”: “USER_IN_ROOM”}</pre>
<i>GET_HISTORY</i>	Cargar mensajes antiguos de un salón.	Cliente	<pre>{“room”: “Manga”, “date”: “2026-05-01”}</pre>	<pre>{“type”: “OK”, “message”: [MESSAGES]}</pre>

<i>HEARTBEAT</i>	Ping de mantenimiento de conexión.	Servidor	<i>{“timestamp”: ...}</i>	No necesita respuesta
------------------	------------------------------------	----------	---------------------------	-----------------------

3 - Manejo de errores

Catálogo mínimo:

- USERNAME_EXISTS
- INVALID_CREDENTIALS
- ROOM_NOT_FOUND
- NOT_IN_ROOM
- MESSAGE_TOO_LONG
- USER_NOT_CONNECTED
- USER_NOT_FOUND
- UNKNOWN_COMMAND
- SERVER_ERROR

4 - Reglas de negocio aplicadas al protocolo

- Límite de caracteres: El campo contenido en los mensajes de salón está limitado a un máximo de 190 caracteres.
- Disponibilidad: Los mensajes privados (*SEND_PRIV_MSG*) solo se entregarán si el destinatario está conectado en ese momento.
- Persistencia: El servidor confirmará la recepción y el almacenamiento de la información relevante antes de su difusión.

5 - Notificaciones de presencia

Cuando un nuevo usuario entra en un salón, el servidor enviará un mensaje automáticamente en el dicho salón.

6 - Heartbeat y desconexión

En el servidor, un hilo comprueba cada minuto si algún usuario lleva más de 5 minutos sin enviar un heartbeat, en ese caso, considera que el cliente se ha desconectado.

7 - Persistencia

Persistir:

- Usuarios (*username, password*)
- Mensajes por salón
- Salones

No persistir:

- Mensajes privados
- Estado de conexión

8 - Posible mejora (opcional)

- Enviar solo nuevos mensajes en lugar de todo el histórico
- Añadir *MESSAGE_ID*
- Compresión simple